System Design Master Template

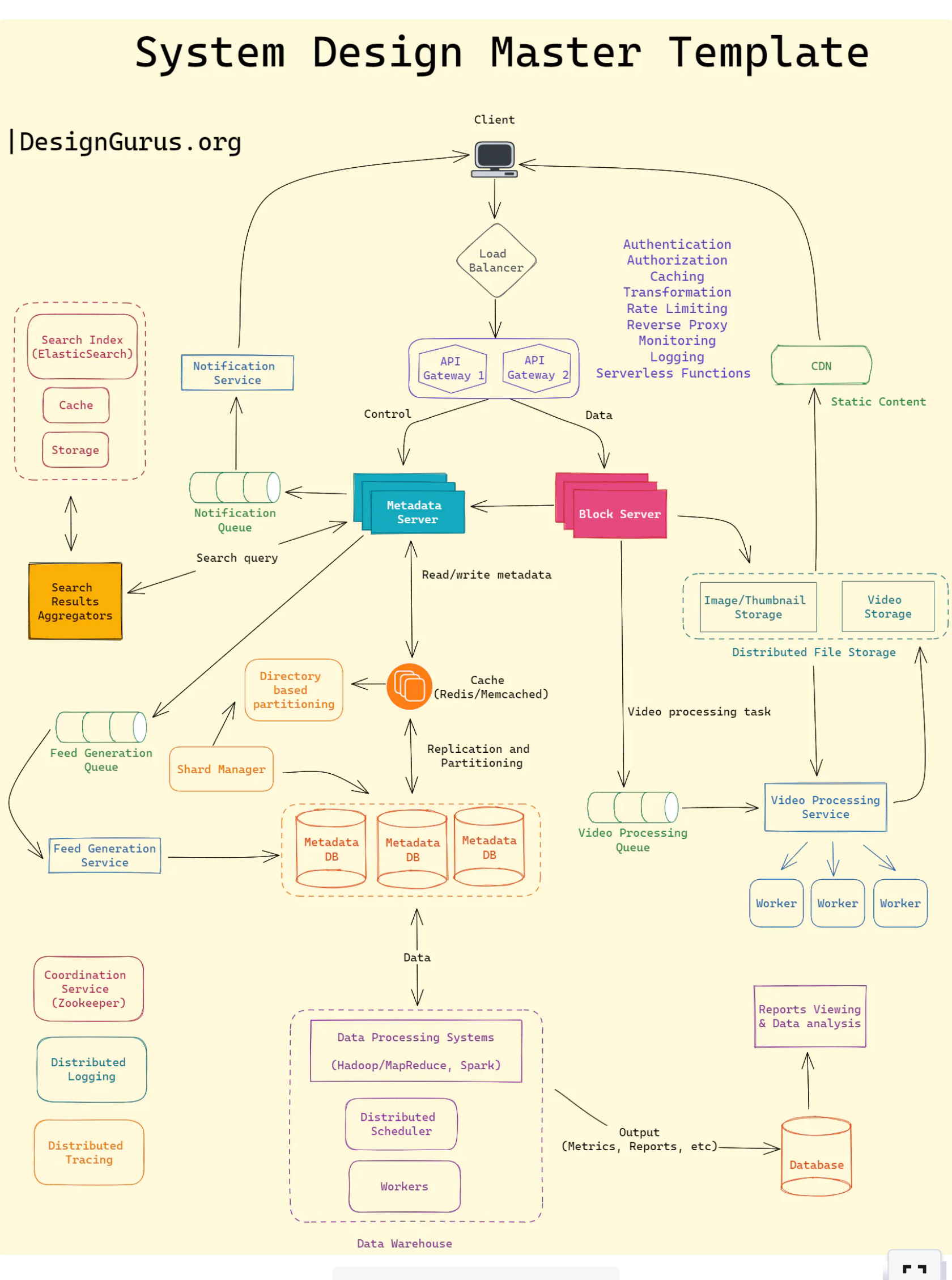
System design interviews are unstructured by design. In these interviews, you are asked to take on an open-ended design problem that doesn't have a standard solution.

The two biggest challenges of answering a system design interview question are:

1. To know where to start.
2. To know if you have talked about all the important parts of the system.

To simplify this process, this course offers a comprehensive system design template that can effectively guide you in addressing any system design interview question.

Have a look at the following image to understand the major components that could be part of any system design and how these components interact with each other.


System Design Master Template
|DesignGurus.org
Client
Load Balancer
Authentication
Authorization
Caching
Transformation
Rate Limiting
Reverse Proxy
Monitoring
Logging
Serverless Functions
API Gateway 1
API Gateway 2
CDN
Static Content
Search Index (ElasticSearch)
Cache
Storage
Notification Service
Notification Queue
Control
Data
Metadata Server
Block Server
Search query
Search Results Aggregators
Read/write metadata
Image/Thumbnail Storage
Video Storage
Distributed File Storage
Directory based partitioning
Cache (Redis/Memcached)
Video processing task
Feed Generation Queue
Shard Manager
Replication and Partitioning
Feed Generation Service
Metadata DB
Metadata DB
Metadata DB
Video Processing Queue
Video Processing Service
Worker
Worker
Worker
Coordination Service (Zookeeper)
Distributed Logging
Distributed Tracing
Data
Data Processing Systems (Hadoop/MapReduce, Spark)
Distributed Scheduler
Workers
Data Warehouse
Output (Metrics, Reports, etc)
Reports Viewing & Data analysis
Database

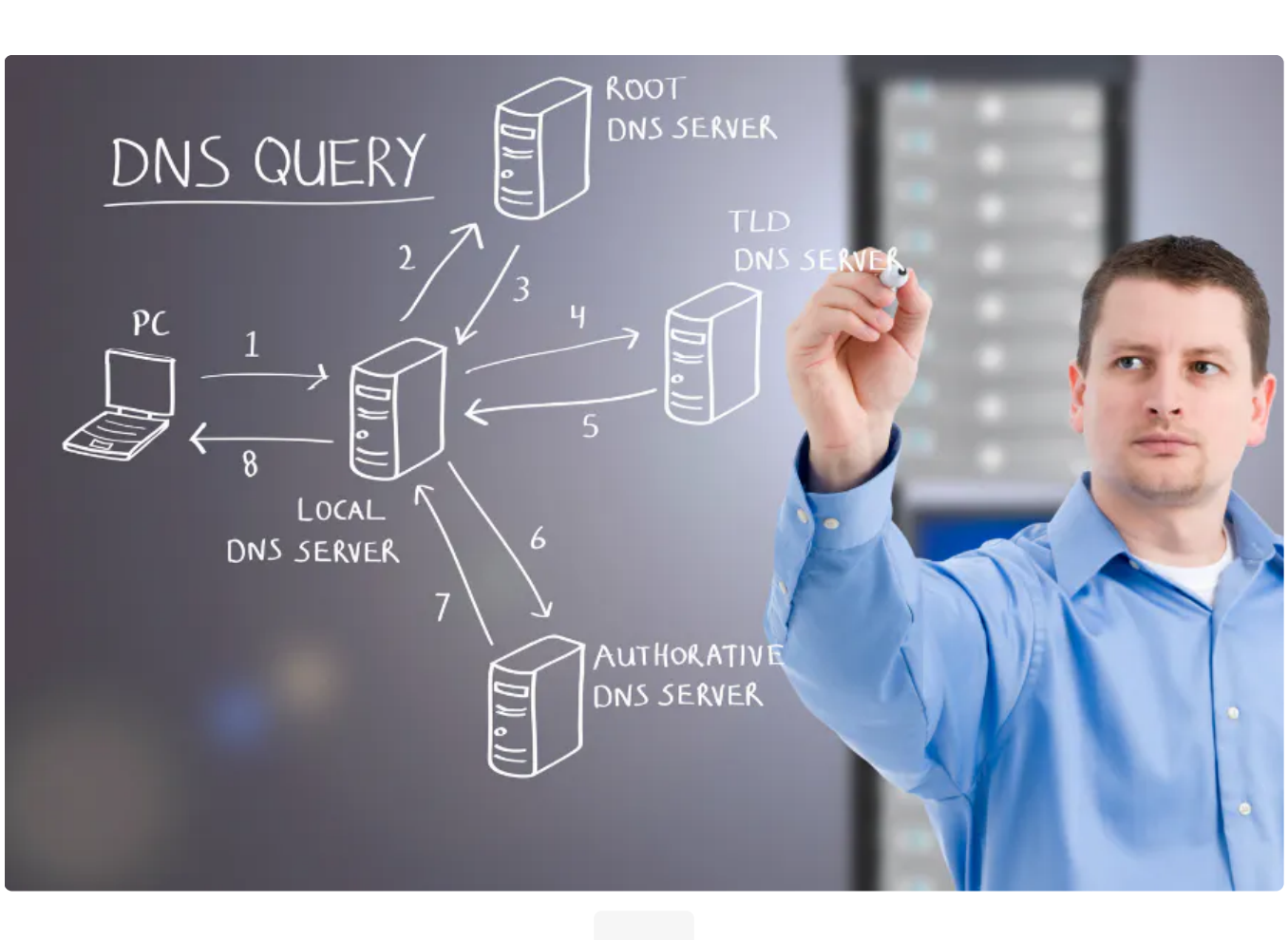
System Design Master Template

With this master template in mind, we will discuss the 18 essential system design concepts. Here is a brief description of each:

1. Domain Name System (DNS)

The Domain Name System (DNS) serves as a fundamental component of the internet infrastructure, translating user-friendly domain names into their corresponding IP addresses. It acts as a phonebook for the internet, enabling users to access websites and services by entering easily memorable domain names, such as www.designgurus.io, rather than the numerical IP addresses like "192.0.2.1" that computers utilize to identify each other.

When you input a domain name into your web browser, the DNS is responsible for finding the associated IP address and directing your request to the appropriate server. This process commences with your computer sending a query to a recursive resolver, which then searches a series of DNS servers, beginning with the root server, followed by the Top-Level Domain (TLD) server, and ultimately the authoritative name server. Once the IP address is located, the recursive resolver returns it to your computer, allowing your browser to establish a connection with the target server and access the desired content.


DNS QUERY
ROOT DNS SERVER
TLD DNS SERVER
PC
1
2
3
4
5
6
7
8
LOCAL DNS SERVER
AUTHORATIVE DNS SERVER

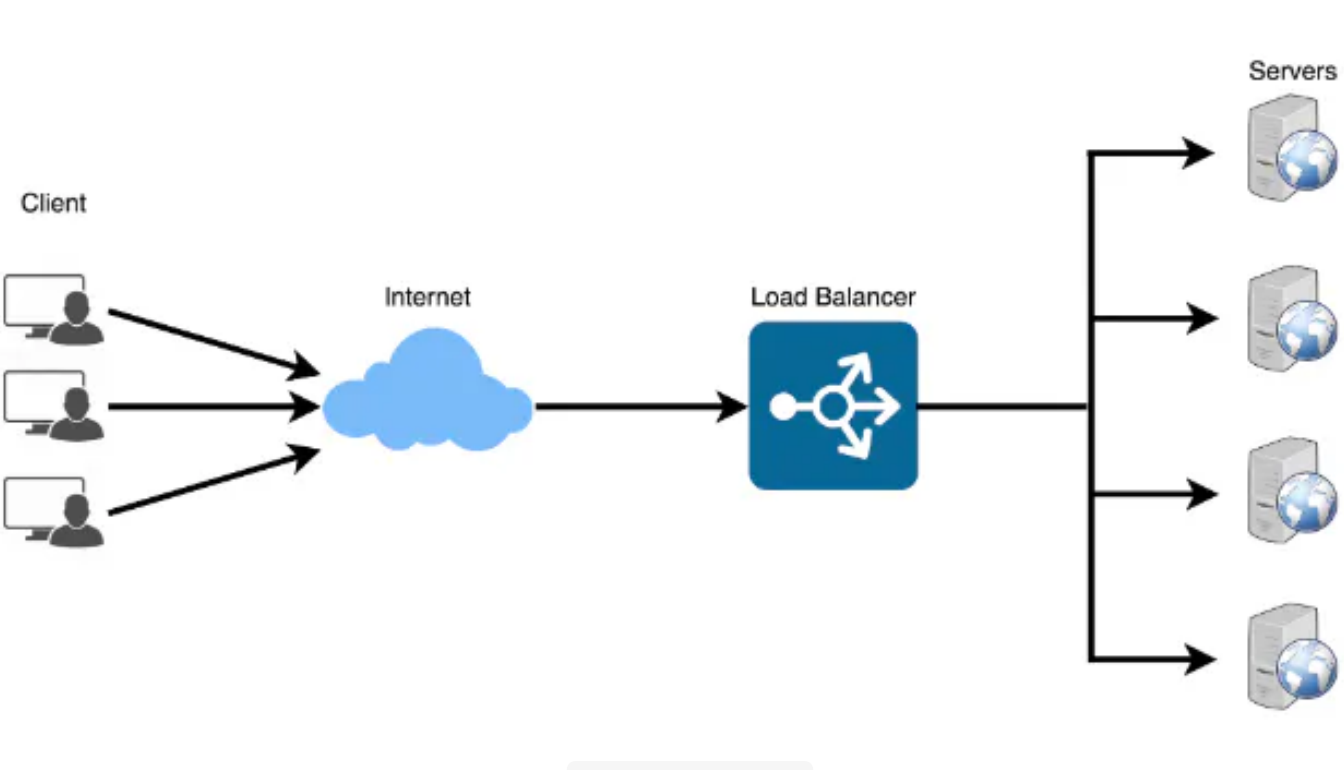
DNS

2. Load Balancer

A load balancer is a networking device or software designed to distribute incoming network traffic across multiple servers, ensuring optimal resource utilization, reduced latency, and maintained high availability. It plays a crucial role in scaling applications and efficiently managing server workloads, particularly in situations where there is a sudden surge in traffic or uneven distribution of requests among servers.

Load balancers employ various algorithms to determine the distribution of incoming traffic. Some common algorithms include:

- **Round Robin**: Requests are sequentially and evenly distributed across all available servers in a cyclical manner.
- **Least Connections**: The load balancer assigns requests to the server with the fewest active connections, giving priority to less-busy servers.
- **IP Hash**: The client's IP address is hashed, and the resulting value is used to determine which server the request should be directed to. This method ensures that a specific client's requests are consistently routed to the same server, helping maintain session persistence.


Client
Internet
Load Balancer
Servers

Load Balancer

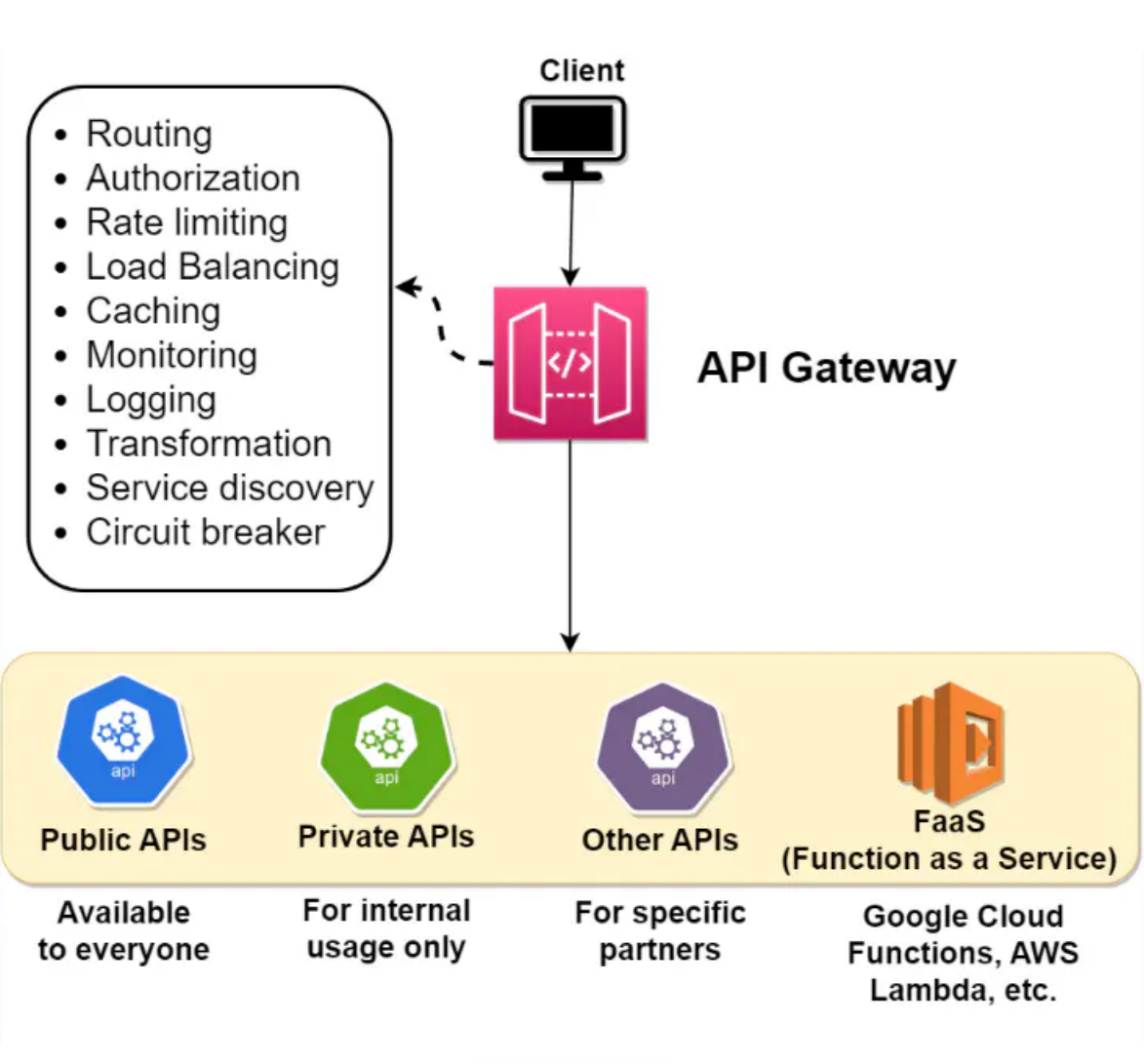
3. API Gateway

An API Gateway serves as a server or service that functions as an intermediary between external clients and the internal microservices or API-based backend services of an application. It is a vital component in contemporary architectures, particularly in microservices-based systems, where it streamlines the communication process and offers a single entry point for clients to access various services.

The primary functions of an API Gateway encompass:

1. Request Routing: The API Gateway directs incoming API requests from clients to the appropriate backend service or microservice, based on predefined rules and configurations.
2. Authentication and Authorization: The API Gateway manages user authentication and authorization, ensuring that only authorized clients can access the services. It verifies API keys, tokens, or other credentials before routing requests to the backend services.
3. Rate Limiting and Throttling: To safeguard backend services from excessive load or abuse, the API Gateway enforces rate limits or throttles requests from clients according to predefined policies.
4. Caching: In order to minimize latency and backend load, the API Gateway caches frequently-used responses, serving them directly to clients without the need to query the backend services.
5. Request and Response Transformation: The API Gateway can modify requests and responses, such as converting data formats, adding or removing headers, or altering query parameters, to ensure compatibility between clients and services.

Is it Possible to Use a Load Balancer and an API Gateway Together? Yes, in many real-world architectures, both of these components work together, where the Load Balancer effectively manages traffic across multiple instances of API Gateways or directly to services, depending on the setup. See details here.

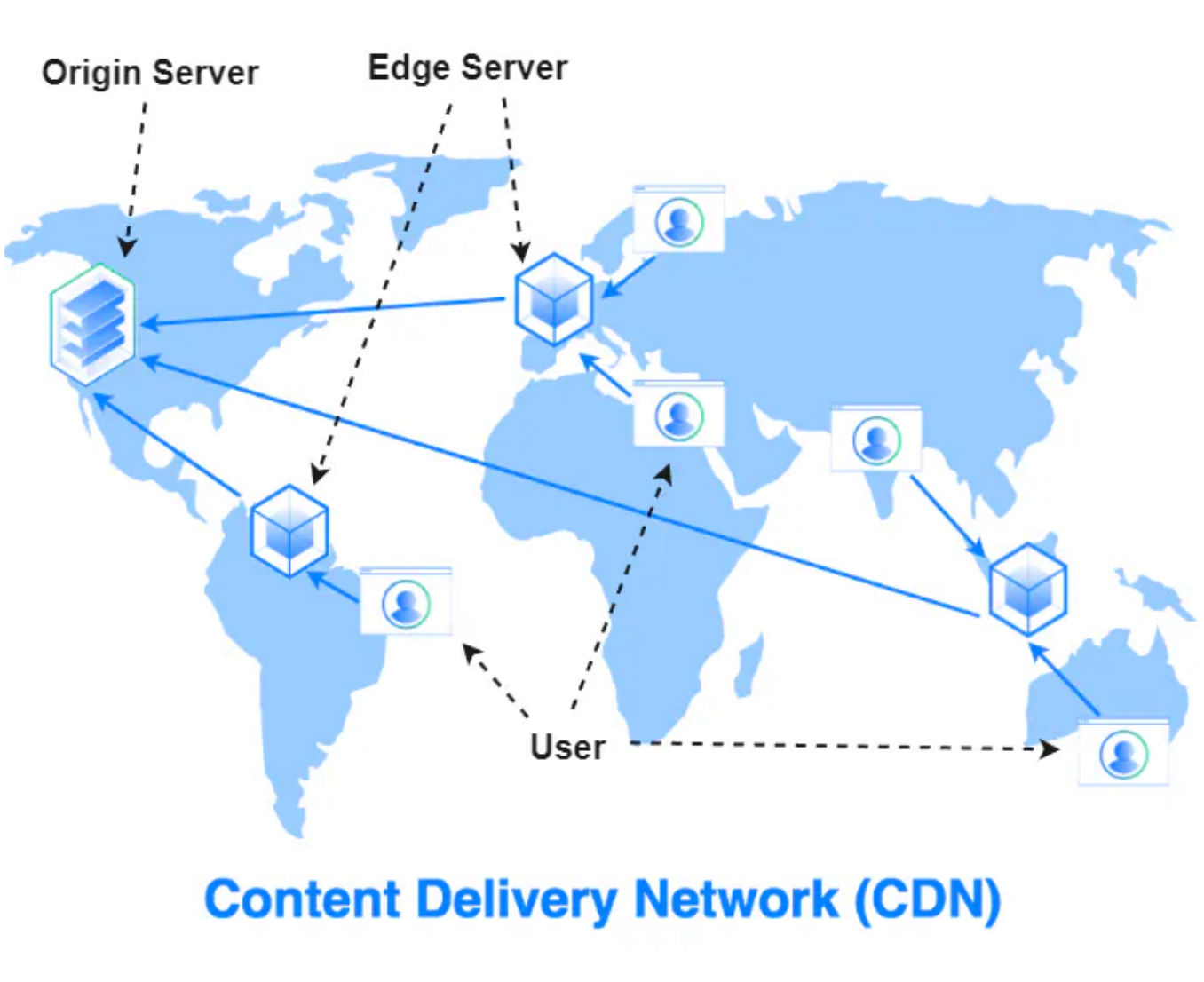

Routing
Authorization
Rate limiting
Load Balancing
Caching
Monitoring
Logging
Transformation
Service discovery
Circuit breaker
Client
API Gateway
Public APIs
Private APIs
Other APIs
FaaS (Function as a Service)
Available to everyone
For internal usage only
For specific partners
Google Cloud Functions, AWS Lambda, etc.

API Gateway

4. CDN

A Content Delivery Network (CDN) is a distributed network of servers that store and deliver content, such as images, videos, stylesheets, and scripts, to users from locations that are geographically closer to them. CDNs are designed to enhance the performance, speed, and reliability of content delivery to end-users, irrespective of their location relative to the origin server. Here's how a CDN operates:

1. When a user requests content from a website or application, the request is directed to the nearest CDN server, also known as an edge server.
2. If the edge server has the requested content cached, it directly serves the content to the user. This process reduces latency and improves the user experience, as the content travels a shorter distance.
3. If the content is not cached on the edge server, the CDN retrieves it from the origin server or another nearby CDN server. Once the content is fetched, it is cached on the edge server and served to the user.
4. To ensure the content stays up-to-date, the CDN periodically checks the origin server for changes and updates its cache accordingly.

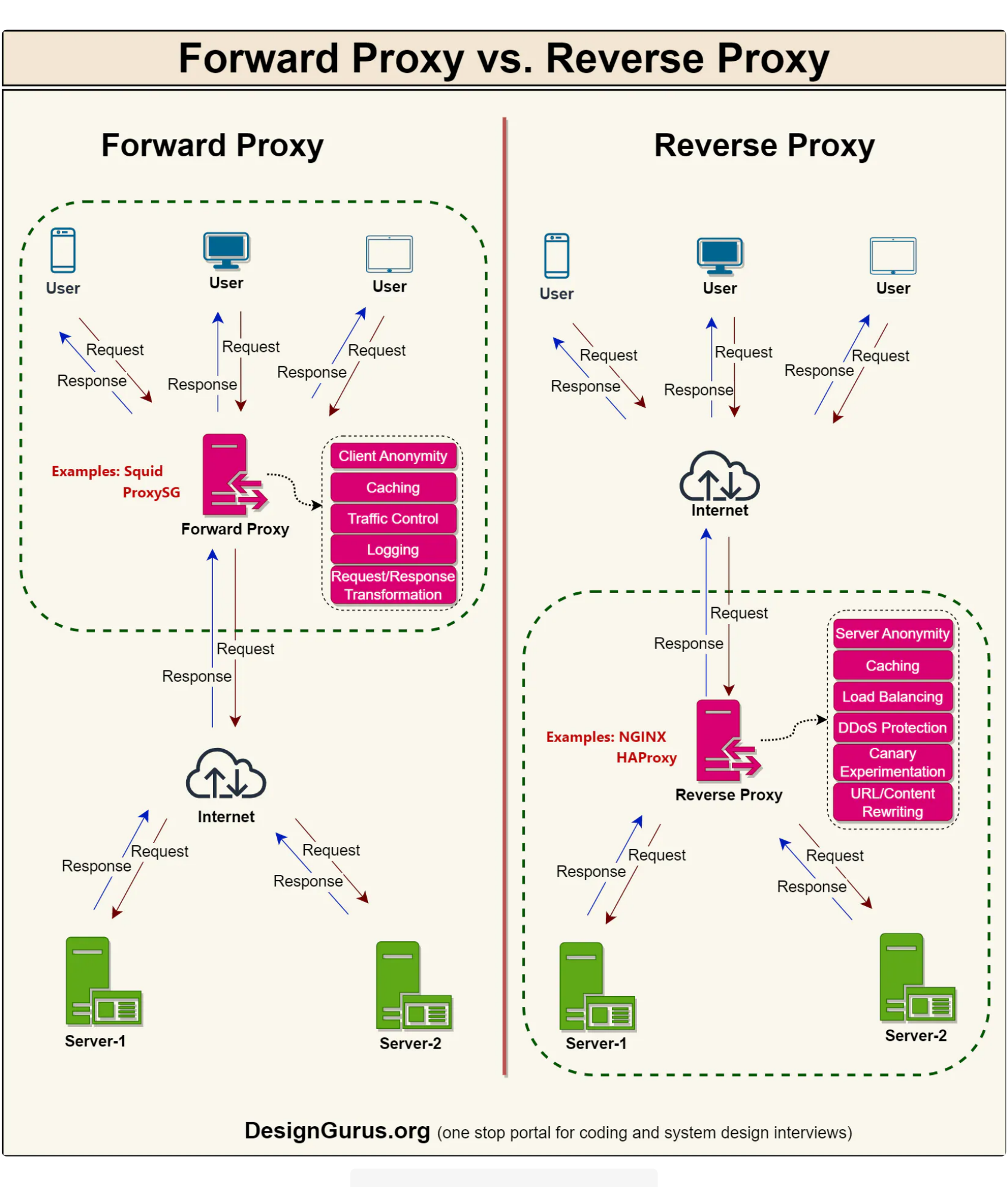

Origin Server
Edge Server
User
Content Delivery Network (CDN)

CDN

5. Forward Proxy vs. Reverse Proxy

A forward proxy, also referred to as a "proxy server" or simply "proxy," is a server positioned in front of one or more client machines, acting as an intermediary between the clients and the internet. When a client machine requests a resource on the internet, the request is initially sent to the forward proxy. The forward proxy then forwards the request to the internet on behalf of the client machine and returns the response to the client machine.

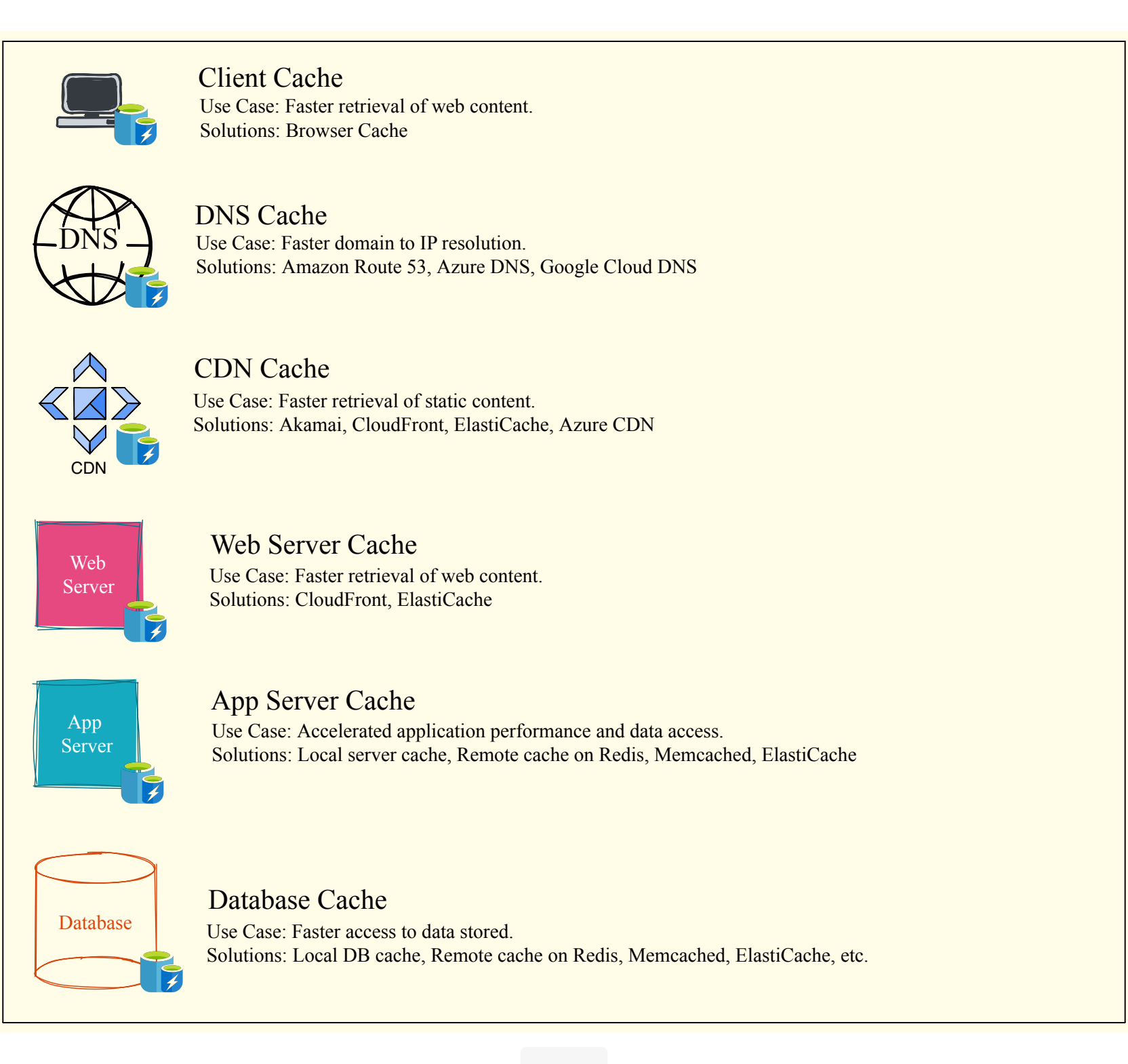
On the other hand, a reverse proxy is a server that sits in front of one or more web servers, serving as an intermediary between the web servers and the internet. When a client requests a resource on the internet, the request is first sent to the reverse proxy. The reverse proxy then forwards the request to one of the web servers, which returns the response to the reverse proxy. Finally, the reverse proxy returns the response to the client.


Forward Proxy vs. Reverse Proxy
Forward Proxy
User
User
User
Request
Response
Examples: Squid ProxySG
Forward Proxy
Client Anonymity
Caching
Traffic Control
Logging
Request/Response Transformation
Internet
Server-1
Server-2
Reverse Proxy
Internet
Examples: NGINX HAProxy
Reverse Proxy
Server Anonymity
Caching
Load Balancing
DDoS Protection
Canary Experimentation
URL/Content Rewriting
Server-1
Server-2
DesignGurus.org (one stop portal for coding and system design interviews)

Forward Proxy vs. Reverse Proxy

6. Caching

Cache is a high-speed storage layer positioned between the application and the original data source, such as a database, file system, or remote web service. When an application requests data, the cache is checked first. If the data is present in the cache, it is returned to the application. If the data is not found in the cache, it is retrieved from its original source, stored in the cache for future use, and then returned to the application. In a distributed system, caching can occur in multiple locations, including the client, DNS, CDN, load balancer, API gateway, server, database, and more.


DNS
CDN
Web Server
App Server
Database

Client Cache
Use Case: Faster retrieval of web content.
Solutions: Browser Cache

DNS Cache
Use Case: Faster domain to IP resolution.
Solutions: Amazon Route 53, Azure DNS, Google Cloud DNS

CDN Cache
Use Case: Faster retrieval of static content.
Solutions: Akamai, CloudFront, ElastiCache, Azure CDN

Web Server Cache
Use Case: Faster retrieval of web content.
Solutions: CloudFront, ElastiCache

App Server Cache
Use Case: Accelerated application performance and data access.
Solutions: Local server cache, Remote cache on Redis, Memcached, ElastiCache

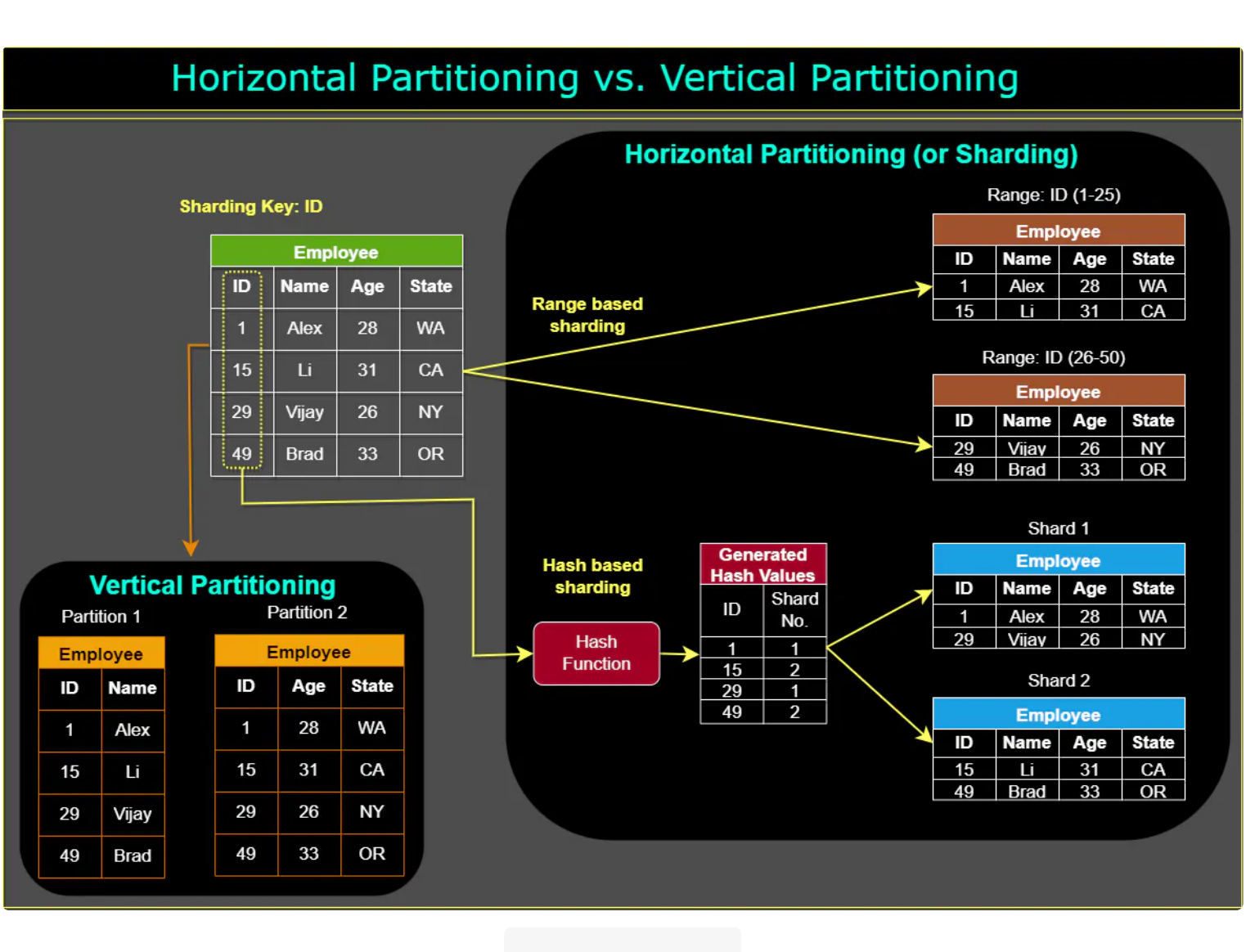
Database Cache
Use Case: Faster access to data stored.
Solutions: Local DB cache, Remote cache on Redis, Memcached, ElastiCache, etc.

Cache

7. Data Partitioning

In a database, **horizontal partitioning**, often referred to as **sharding**, entails dividing the rows of a table into smaller tables and storing them on distinct servers or database instances. This method is employed to distribute the database load across multiple servers, thereby enhancing performance.

Conversely, **vertical partitioning** involves splitting the columns of a table into separate tables. This technique aims to reduce the column count in a table and boost the performance of queries that only access a limited number of columns.


Horizontal Partitioning vs. Vertical Partitioning
Horizontal Partitioning (or Sharding)
Sharding Key: ID
Employee
ID Name Age State
1 Alex 28 WA
15 Li 31 CA
29 Vijay 26 NY
49 Brad 33 OR
Range based sharding
Range: ID (1-25)
Employee
ID Name Age State
1 Alex 28 WA
15 Li 31 CA
Range: ID (26-50)
Employee
ID Name Age State
29 Vijay 26 NY
49 Brad 33 OR
Hash based sharding
Hash Function
Generated Hash Values
ID Shard No.
1 1
15 2
29 1
49 2
Shard 1
Employee
ID Name Age State
1 Alex 28 WA
29 Vijay 26 NY
Shard 2
Employee
ID Name Age State
15 Li 31 CA
49 Brad 33 OR
Vertical Partitioning
Partition 1
Employee
ID Name
1 Alex
15 Li
29 Vijay
49 Brad
Partition 2
Employee
ID Age State
1 28 WA
15 31 CA
29 26 NY
49 33 OR

Data partitioning

8. Database Replication

Database replication is a method employed to maintain multiple copies of the same database across various servers or locations. The main objective of database replication is to enhance data availability, redundancy, and fault tolerance, ensuring the system remains operational even in the face of hardware failures or other issues.

In a replicated database configuration, one server serves as the primary (or master) database, while others act as replicas (or slaves). This process involves synchronizing data between the primary database and replicas, ensuring all possess the same up-to-date information. Database replication provides several advantages, including:

1. Improved Performance: By distributing read queries among multiple replicas, the load on the primary database can be reduced, leading to faster query response times.
2. High Availability: If the primary database experiences failure or downtime, replicas can continue to provide data, ensuring uninterrupted access to the application.
3. Enhanced Data Protection: Maintaining multiple copies of the database across different locations helps safeguard against data loss due to hardware failures or other disasters.
4. Load Balancing: Replicas can handle read queries, allowing for better load distribution and reducing overall stress on the primary database.

9. Distributed Messaging Systems

Distributed messaging systems provide a reliable, scalable, and fault-tolerant means for exchanging messages between numerous, possibly geographically-dispersed applications, services, or components. These systems facilitate communication by decoupling sender and receiver components, enabling them to develop and function independently. Distributed messaging systems are especially valuable in large-scale or intricate systems, like those seen in microservices architectures or distributed computing environments. Examples of these systems include Apache Kafka and RabbitMQ.

10. Microservices

Microservices represent an architectural style wherein an application is organized as an assembly of small, loosely-coupled, and autonomously deployable services. Each microservice is accountable for a distinct aspect of functionality or domain within the application and communicates with other microservices via well-defined APIs. This method deviates from the conventional monolithic architecture, where an application is constructed as a single, tightly-coupled unit.

The primary characteristics of microservices include:

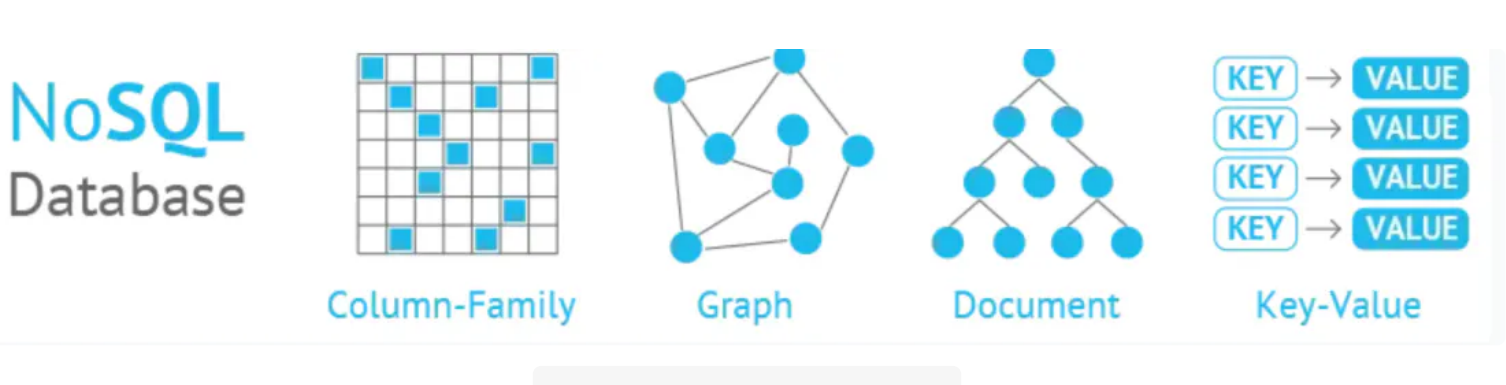
1. Single Responsibility: Adhering to the Single Responsibility Principle, each microservice focuses on a specific function or domain, making the services more straightforward to comprehend, develop, and maintain.
2. Independence: Microservices can be independently developed, deployed, and scaled, offering increased flexibility and agility in the development process. Teams can work on various services simultaneously without impacting the entire system.
3. Decentralization: Typically, microservices are decentralized, with each service possessing its data and business logic. This approach fosters separation of concerns and empowers teams to make decisions and select technologies tailored to their unique requirements.
4. Communication: Microservices interact with each other using lightweight protocols, such as HTTP/REST, gRPC, or message queues. This fosters interoperability and facilitates the integration of new services or the replacement of existing ones.
5. Fault Tolerance: As microservices are independent, the failure of one service does not necessarily result in the collapse of the entire system, enhancing the application's overall resiliency.

11. NoSQL Databases

NoSQL databases, or "Not Only SQL" databases, are non-relational databases designed to store, manage, and retrieve unstructured or semi-structured data. They offer an alternative to traditional relational databases, which rely on structured data and predefined schemas. NoSQL databases have become popular due to their flexibility, scalability, and ability to handle large volumes of data, making them well-suited for modern applications, big data processing, and real-time analytics.

NoSQL databases can be categorized into four main types:

1. Document-Based: These databases store data in document-like structures, such as JSON or BSON. Each document is self-contained and can have its own unique structure, making them suitable for handling heterogeneous data. Examples of document-based NoSQL databases include MongoDB and Couchbase.
2. Key-Value: These databases store data as key-value pairs, where the key acts as a unique identifier, and the value holds the associated data. Key-value databases are highly efficient for simple read and write operations, and they can be easily partitioned and scaled horizontally. Examples of key-value NoSQL databases include Redis and Amazon DynamoDB.
3. Column-Family: These databases store data in column families, which are groups of related columns. They are designed to handle write-heavy workloads and are highly efficient for querying data with a known row and column keys. Examples of column-family NoSQL databases include Apache Cassandra and HBase.
4. Graph-Based: These databases are designed for storing and querying data that has complex relationships and interconnected structures, such as social networks or recommendation systems. Graph databases use nodes, edges, and properties to represent and store data, making it easier to perform complex traversals and relationship-based queries. Examples of graph-based NoSQL databases include Neo4j and Amazon Neptune.


NoSQL Database
Column-Family
Graph
Document
KEY → VALUE
KEY → VALUE
KEY → VALUE
KEY → VALUE
Key-Value

Types of NoSQL databases

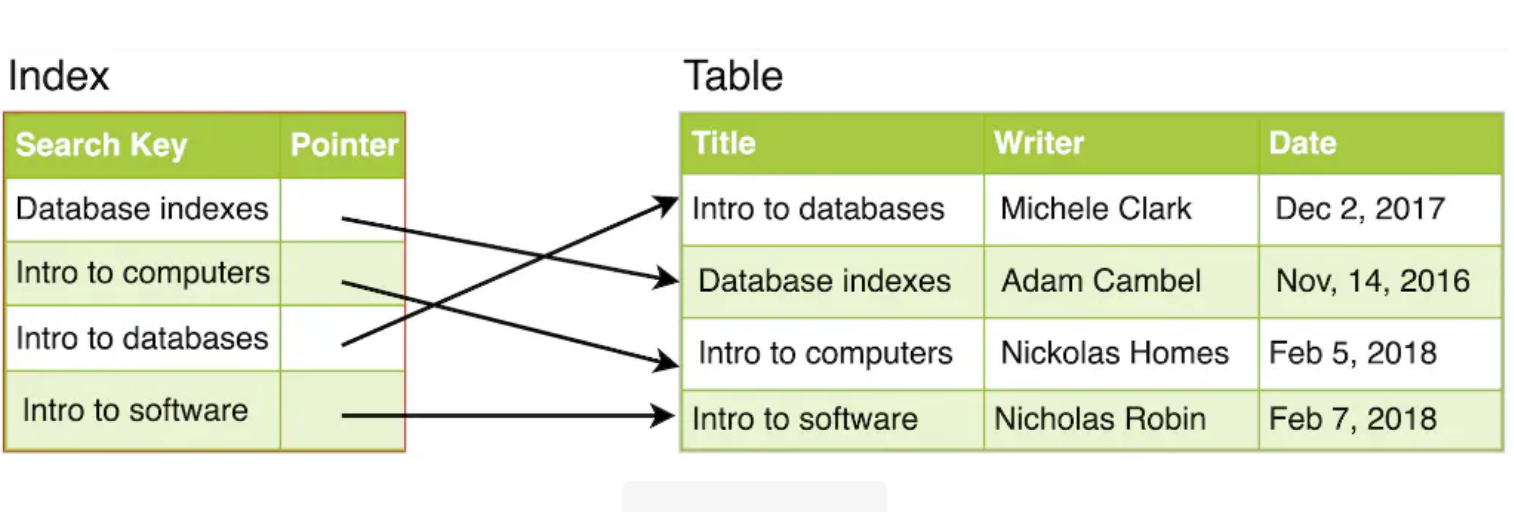
12. Database Index

Database indexes are data structures that enhance the speed and efficiency of query operations within a database. They function similarly to an index in a book, enabling the database management system (DBMS) to swiftly locate data associated with a specific value or group of values, without the need to search through every row in a table. By offering a more direct route to the desired data, indexes can considerably decrease the time required to retrieve information from a database.

Indexes are typically constructed on one or more columns of a database table. The B-tree index is the most prevalent type, organizing data in a hierarchical tree structure, which allows for rapid search, insertion, and deletion operations. Other types of indexes, such as bitmap indexes and hash indexes, exist as well, each with their particular use cases and advantages.

Although indexes can significantly enhance query performance, they also involve certain trade-offs:

- **Storage Space**: Indexes require additional storage space since they generate and maintain separate data structures alongside the original table data.
- **Write Performance**: When data is inserted, updated, or deleted in a table, the corresponding indexes must also be updated, which may slow down write operations.


Index
Search Key
Pointer
Database indexes
Intro to computers
Intro to databases
Intro to software
Table
Title
Writer
Date
Intro to databases
Michele Clark
Dec 2, 2017
Database indexes
Adam Cambel
Nov, 14, 2016
Intro to computers
Nickolas Homes
Feb 5, 2018
Intro to software
Nicholas Robin
Feb 7, 2018

Database Index

13. Distributed File Systems

Distributed file systems are storage systems designed to manage and grant access to files and directories across multiple servers, nodes, or machines, frequently distributed across a network. They allow users and applications to access and modify files as though they were situated on a local file system, despite the fact that the actual files may be physically located on various remote servers. Distributed file systems are commonly employed in large-scale or distributed computing environments to offer fault tolerance, high availability, and enhanced performance.

14. Notification System

These are used to send notifications or alerts to users, such as emails, push notifications, or text messages.